Lecture 9 · Part II · Extended notes

ML system design interview: harmful content detection

Dataset: A mock interview, no dataset

Why these notes exist

This lecture sits outside Géron, the course's primary text. For it *these notes are the primary source*, and it is examined on exactly the same terms as the textbook lectures. Everything in the slide deck is here, with the arguments written out at length rather than compressed onto a slide; the numbers are the ones the accompanying notebook prints, and every one of them is a measurement on the dataset named above rather than a figure quoted from a paper.

Contents

1 The format **2**

2 The prompt **3**

2.1 Step 1 — Clarify . . . 3
2.2 Step 2 — Frame the task . . . 3
2.3 Step 3 — Metrics . . . 3
2.4 Step 4 — Data and labels . . . 4
2.5 Step 5 — Features . . . 4
2.6 Step 6 — The model . . . 4
2.7 Step 7 — Thresholds and offline evaluation . . . 5
2.8 Step 8 — Serving . . . 5
2.9 Step 9 — Launch, monitoring, iteration . . . 5

3 Follow-ups, red flags, and practice **5**

4 Exercises **6**

1 The format

An ML system design interview is a standard round for machine learning engineers and applied scientists at the large technology companies and at most companies that ship models. The candidate receives one open-ended prompt — "design the system that . . . " — and a whiteboard. There is no single right answer. The candidate drives; the interviewer plays the product manager, answers questions, adds constraints, and pushes on whatever looks weak. What is tested is whether a vague business problem can be turned into a system that works, can be measured, and can be run.

Signal	A weak answer	A strong answer
Framing	"train a classifier"	asks what the output is used for, and what each mistake costs
Metrics	accuracy, or AUC alone	the metric the decision needs, offline and online
Data	assumes clean labels exist	says where labels come from, and how they are biased
Modelling	names the biggest model	a baseline first, and a reason for every step up
Production	stops at training	serving, cost, monitoring, feedback loops
Communication	a list of techniques	trade-offs: "X, because Y; Z would cost W"

A good answer has nine steps: clarify; frame the task; metrics; data and labels; features; the model; training, thresholds and offline evaluation; serving; launch, monitoring and iteration. Cover all nine briefly before going deep anywhere, then go deep where the interviewer pulls: running out of time half-way is the most common way to fail.

How the lecture runs

The lecturer is the interviewer and the class, in pairs, is the candidate. At every step the pairs write an answer first; then a strong answer and the red flags an interviewer would note. The strong answer is one good answer, not the only one.

2 The prompt

"We run a social platform with billions of users. Design a system that detects harmful content."

2.1 Step 1 — Clarify

Ask the questions that change the design, write the answers on the board as assumptions, and commit. Which harms (each policy is a label, a threshold and a team)? Which content and

modalities (they decide features and cost)? How much, in how many languages (the cost of a prediction, and where the model is blind)? Before publishing or after (the latency budget)? What happens to flagged content (the output is an action, not a label)? Which labels exist? Appeals, regulators, transparency reports?

The interviewer's answers used in the lecture: five harms — violence, nudity, hate speech, harassment, self-harm — each with its own written policy; posts with text and images, hundreds of millions a day in dozens of languages; a check at upload within tens of milliseconds and again as a post spreads; remove automatically when sure, send to a human reviewer when unsure, reduce distribution in between; labels from past reviewer decisions and user reports; users can appeal; a regulator expects a public report.

Red flags

Designing before asking; or asking twenty questions and never committing.

2.2 Step 2 — Frame the task

Input: a post — text, image, author, where and when it was posted. Output: one probability per harm, $p(\text{violence} \mid \text{post})$, $p(\text{hate} \mid \text{post})$, ... The task is **multi-label**: a post can be violent and hateful at once, so the output layer is one sigmoid per harm (Lecture 7), not a softmax, which forces the probabilities to sum to one. A separate **policy layer** turns the probabilities into actions with a threshold per harm and per action, so that a policy change does not require retraining.

Of the three architectures — one binary model, one model per harm, one shared model with a head per harm — the shared model is the usual choice: rare harms borrow the representation learned from common ones, and each post needs one pass. The price is coupling: every head must be re-evaluated on every release.

2.3 Step 3 — Metrics

Violating posts are a fraction of a percent of all posts, so accuracy is meaningless and the ROC curve flatters (Lecture 3). Offline, per harm: the precision–recall curve and the area under it, read at the operating points the actions need — recall at the precision required for automatic removal, and at the precision reviewers can afford — and calibration, because the policy layer and the queue size use the outputs as probabilities (Lecture 4). Report each harm separately: missing self-harm and missing spam do not cost the same.

Online: the **prevalence** of each harm — the share of views that land on violating content, estimated by reviewers labelling a random sample of what was viewed — the proactive rate (removed before any report), appeals overturned (a precision signal from the field), the reviewer queue, and guardrails such as creators' complaints. "Number of posts removed" is not a goal: it rewards removing everything.

2.4 Step 4 — Data and labels

Reviewer decisions are high quality but expensive and exist only for what was flagged — **selection bias**. User reports are cheap but noisy, open to mass-reporting campaigns, and skewed towards

popular posts. Appeal outcomes are the hardest cases but few. Matching against known-bad images catches only what has been seen before. Measure reviewer agreement on a doubly labelled slice: it is the ceiling for any model. Store the policy version with every label, because a label is valid only for the policy it was given under.

Label a random sample of all traffic every week: it is the only unbiased estimate of prevalence and of what the model misses. For training, label where a few labels buy the most — posts the model is least sure about and representatives of new clusters, as with Lecture 6's forty labels. Split by time (train on the past, test on later weeks) and by author, and remove near-duplicate reposts across the split; a random split of posts lets the model memorise authors and viral images.

2.5 Step 5 — Features

Text and image enter as vectors from pretrained encoders (opened later in the course; here black boxes), plus the text read out of images; the author (account age, past violations, followers); the context (where, which group, language, hour); and the reaction (reports, shares, spread, comments). The first four exist at upload; the reaction does not. Training the upload-time model on "number of reports" is **leakage in time**: compute every feature as of the moment of the decision, and re-score with reaction features as they arrive.

The data fights back: misspellings, look-alike characters, hateful text inside images, coded words. Character-level features, image text, the author's history and the network of accounts posting the same thing survive this. A benign image with a hateful caption is harmful only in combination.

2.6 Step 6 — The model

Build up and justify each step by the offline metric on an honest split: rules and known-bad matching as the baseline; logistic regression on the frozen vectors and tabular features (Lecture 4); gradient-boosted trees on the same features (Lecture 5); then the shipped model, a network that concatenates text vector, image vector and tabular features (**early fusion**), passes them through shared layers and ends in one sigmoid head per harm, trained with the sum of per-harm binary cross-entropies, weighted so rare harms are not drowned out (Lecture 7–Lecture 8). Fine-tune the encoders only if the gain pays the serving cost. Late fusion — one classifier per modality — misses the benign-image-hateful-caption case. Keep every positive, sample the negatives, and correct the probabilities back with the log-odds shift of Lecture 5's advertising design exercise; check calibration per harm.

2.7 Step 7 — Thresholds and offline evaluation

Each harm has bands: very high probability, remove automatically (the precision the policy team accepts with no human in the loop); high, send to review (reviewer capacity: the expected daily volume must fit the queue); medium, reduce distribution while it is reviewed; low, leave it. Each band is a point on the validation precision–recall curve, signed off as policy: the threshold is a decision, not a hyperparameter (Lecture 3).

Report every metric per language, region, content type and account age, and the worst slice beside the average, with a floor every slice must clear. The known failure is dialects and reclaimed

words flagged as hate speech far more often than standard usage. Read samples of false positives and false negatives before any launch decision.

2.8 Step 8 — Serving

Two stages. At upload, on everything, known-bad matching and a small model within the latency budget: publish what is clearly fine, block what is clearly violating. In the background, the full multimodal model on the uncertain posts, published meanwhile with reduced distribution; re-score as reactions arrive. Order the review queue by expected harm: probability of violation times predicted reach. Compute each post's vectors once and share them. Say the trade-offs: checking before publishing against the delay every user feels; the size of the first-stage model against the latency budget; fail open or fail closed when the model is down, decided in advance; how much reaches humans, against cost and reviewers' wellbeing.

2.9 Step 9 — Launch, monitoring, iteration

Shadow mode first (score, do not act, compare with reviewers); then a gradual ramp; then an A/B test on prevalence and appeals — with the caveat that a control group cannot be left exposed to the worst harms, so for those the random-sample prevalence estimate replaces a pure holdout. Watch flag rates and input distributions per harm and language for new slang, formats and news events; use the random sample and red teams against adapting bad actors; keep labelling the random sample because labels from flagged posts alone never teach the model its misses; relabel when a policy changes. Retrain on a schedule, but release only a model that beats the current one on the latest weeks and clears every slice.

3 Follow-ups, red flags, and practice

Typical follow-ups: a new harm appears overnight (rules and reviewers first, then a new head and targeted labelling); a new language (multilingual encoder, labels in that language, the slice measured before launch); misspellings (character-level and image-text features, adversarial examples); overwhelmed reviewers (order by reach, act automatically only where precision allows); a famous account wrongly removed (appeals, a second review, the same rules for everyone).

Red flags an interviewer writes down: designing before asking; accuracy or one number for everything; free, clean labels; a random split of posts; features that exist only after the decision; the biggest model first; a softmax over harms that co-occur; stopping at training; listing techniques instead of choosing.

To practise, run the nine steps out loud against a clock on products you use: video recommendations, the arrival time of a food delivery, search ranking in a shop, fraud at checkout, "people you may know". Every later lecture ends with a design exercise in this format.

4 Exercises

The five set at the end of this lecture's deck, in the style of the written paper. The solutions are on Lecture 10's deck and in the exercise book.

1. A harmful-content model ends in one softmax over five harms. Say why this is the wrong output layer, and what replaces it. [4]
2. Harmful posts are a fraction of a percent of all posts. Explain why accuracy and the area under the ROC curve both mislead here, and say what you would report offline for each harm. [5]
3. The training labels are past reviewer decisions on posts the current system flagged. Name the bias, its consequence for the next model, and how to correct for it. [5]
4. Adding "number of user reports" sharply raises the offline recall of the model that checks posts at upload. Explain the gain, and say what you would do. [4]
5. For each harm the policy layer has three thresholds: remove automatically, send to a human reviewer, reduce distribution. Say what sets each one. [5]

Summary

A mock ML system design interview on harmful content detection, in the format used for machine learning engineers: clarify before designing; one probability per harm with a separate policy layer; precision and recall per harm at the action thresholds offline, prevalence weighted by views online; labels treated as biased evidence, corrected with random-sample labelling and honest splits; a model built up from rules to a fused network with one head per harm; thresholds set as policy and evaluation per slice; two-stage serving; and a launch through shadow mode and a ramp, watching drift, adversaries and the feedback loop.